



Trabalho Prático: Autómatos Finitos

Detetor de Partículas usando um AFD

U.C. Dados e Computação

João Carlos Monteiro Ferreira

Diana Almeida



Conteúdo

1. Introdução

1.1. Teoria

1.2. Uso no CERN

2. Código

2.1. Estrutura

2.2. Funcionamento

3. Conclusão

4. Referencias



Introdução

Teoria

O nosso autômato A é definido pelo quintuplo $A = (Q, \Sigma, \delta, q_0, F)$, onde:

- **Q (Conjunto de Estados):**

$Q = \{q_{\text{colisao}}, q_{\text{rasto}}, q_{\text{fotao}}, q_{\text{eletrao}}, q_{\text{hadrao}}, q_{\text{muao}}, q_{\text{ruído}}\}$

- **Σ (Alfabeto):**

$\Sigma = \{T, E, H, M\}$

- **q_0 (Estado Inicial):**

$q_{\text{colisão}}$

- **F (Estados Válidos):**

$F = \{q_{\text{fotao}}, q_{\text{eletrao}}, q_{\text{hadrao}}, q_{\text{muao}}\}$

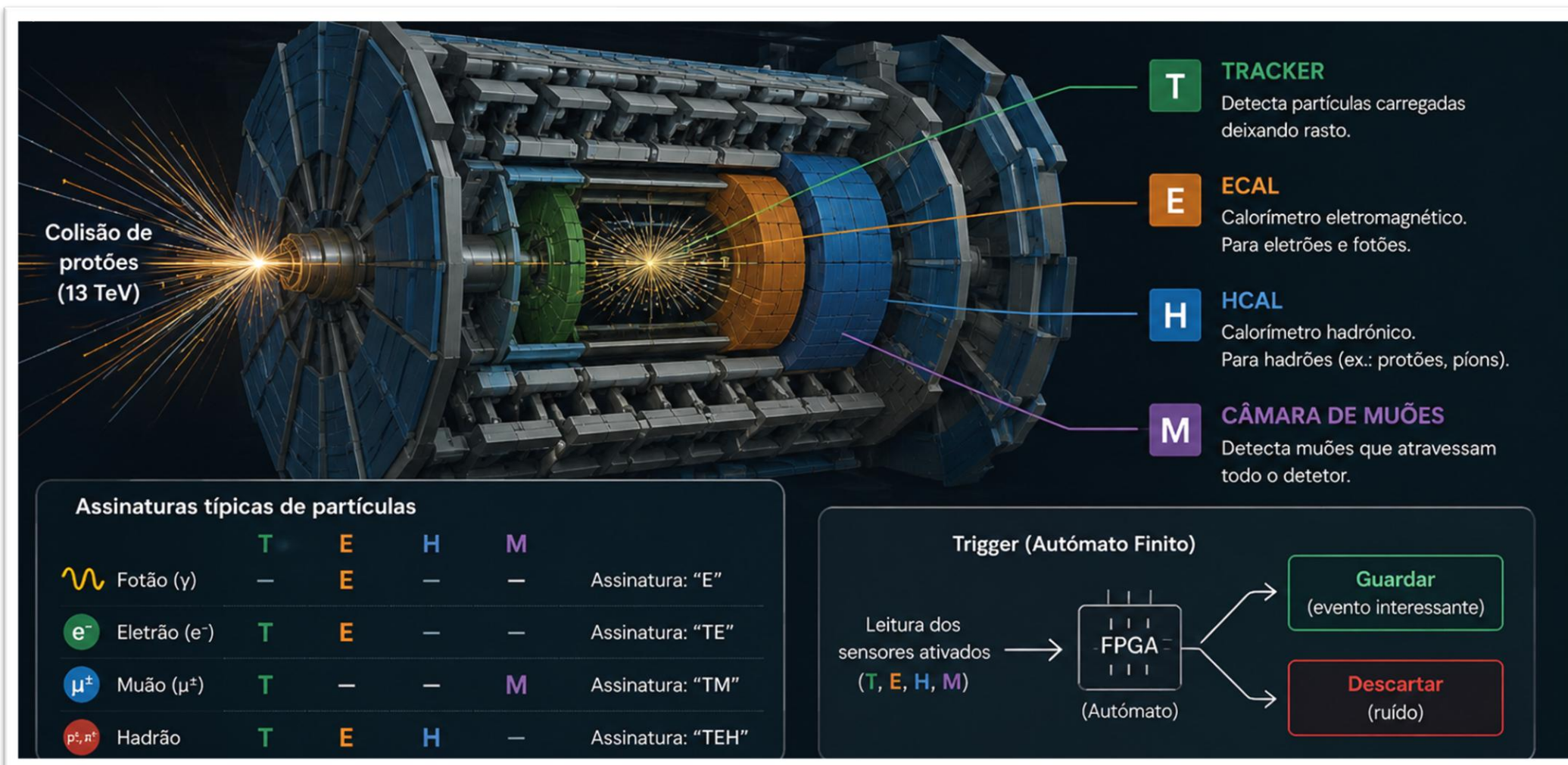
- **δ (Função de Transição):**

Mapeamento $\delta: Q \times \Sigma \rightarrow Q$. Qualquer transição não prevista para uma dada letra conduz ao estado de rejeição $q_{\text{ruído}}$.



Introdução

Uso no CERN





Código

Estrutura

```
class AutomatoDetetorParticulas:
    """...
    def __init__(self):...

    def analisar_assinatura(self, palavra: str, modo_silencioso: bool = False) -> bool: ...

    def simular_colisoes_aleatorias(self, quantidade: int = 3): ...

    def gerar_diagrama_visual(self): ...

if __name__ == "__main__":
    lhc = AutomatoDetetorParticulas()

    diagrama = lhc.gerar_diagrama_visual()
    if diagrama and TEM_DISPLAY:
        display(diagrama)
    elif diagrama:
        diagrama.render('diagrama_automato', view=True)

    lhc.simular_colisoes_aleatorias(3)

#Teste Manual
print(f"\n{'-'*50}\n Modo Manual\n{'-'*50}")
print("Alfabeto disponível: T (Tracker), E (ECAL), H (HCAL), M (Muon)")
print("Escreve uma palavra (ex: 'TE', 'TMM') ou 'sair' para terminar.")

while True:
    try:
        entrada = input("\n Insira os sensores ativados: ").strip()
        if entrada.lower() == 'sair':
            print("Encerrado.")
            break
        if not entrada:
            continue

        lhc.analisar_assinatura(entrada)

    except KeyboardInterrupt:
        print("\n Encerrado.")
        break
```



Código

Funcionamento

```
class AutomatoDetetorParticulas:
    """
    Classe que modela um AFD (A = (Q, Σ, δ, q0, F))
    para validar e classificar partículas em detetores.
    """
    def __init__(self):
        # (Alfabeto): Sensores disponíveis
        self.alfabeto = ['T', 'E', 'H', 'M']

        # q0 (Estado Inicial)
        self.estado_inicial = 'q_colisao'

        # F (Conjunto de Estados de Aceitação) mapeados para a respetiva partícula
        self.estados_aceitacao = {
            'q_fotao': 'Fotão (γ)',
            'q_eletrao': 'Eletrão (e-)',
            'q_hadrao': 'Hadrão (p+)',
            'q_muao': 'Muão (μ-)'
        }

        # (Função de Transição): Dicionário [estado_atual][simbolo_lido] = estado_seguente
        self.transicoes = {
            'q_colisao': {'E': 'q_fotao', 'T': 'q_rasto'},
            'q_rasto': {'E': 'q_eletrao', 'H': 'q_hadrao', 'M': 'q_muao'},
            'q_muao': {'M': 'q_muao'}, # O muão atravessa várias câmaras
            'q_fotao': {}, # Estado final (sem saídas)
            'q_eletrao': {}, # Estado final
            'q_hadrao': {}, # Estado final
            'q_ruido': {} # Dead state
        }
```

AutomatoDetetorParticulas – classe principal do programa responsável por representar e controlar o funcionamento do autómato finito determinístico.

```
def analisar_assinatura(self, palavra: str, modo_silencioso: bool = False) -> bool:
    """
    Lê a palavra símbolo a símbolo e transita entre os estados.
    Devolve True se terminar num estado de aceitação, False caso contrário.
    """
    estado_atual = self.estado_inicial

    if not modo_silencioso:
        print(f"\n[A processar palavra: '{palavra}']")

    for sensor in palavra.upper(): # .upper() para garantir que aceita letras minúsculas
        if estado_atual in self.transicoes and sensor in self.transicoes.get(estado_atual, {}):
            estado_atual = self.transicoes[estado_atual][sensor]
        else:
            estado_atual = 'q_ruido'
            break

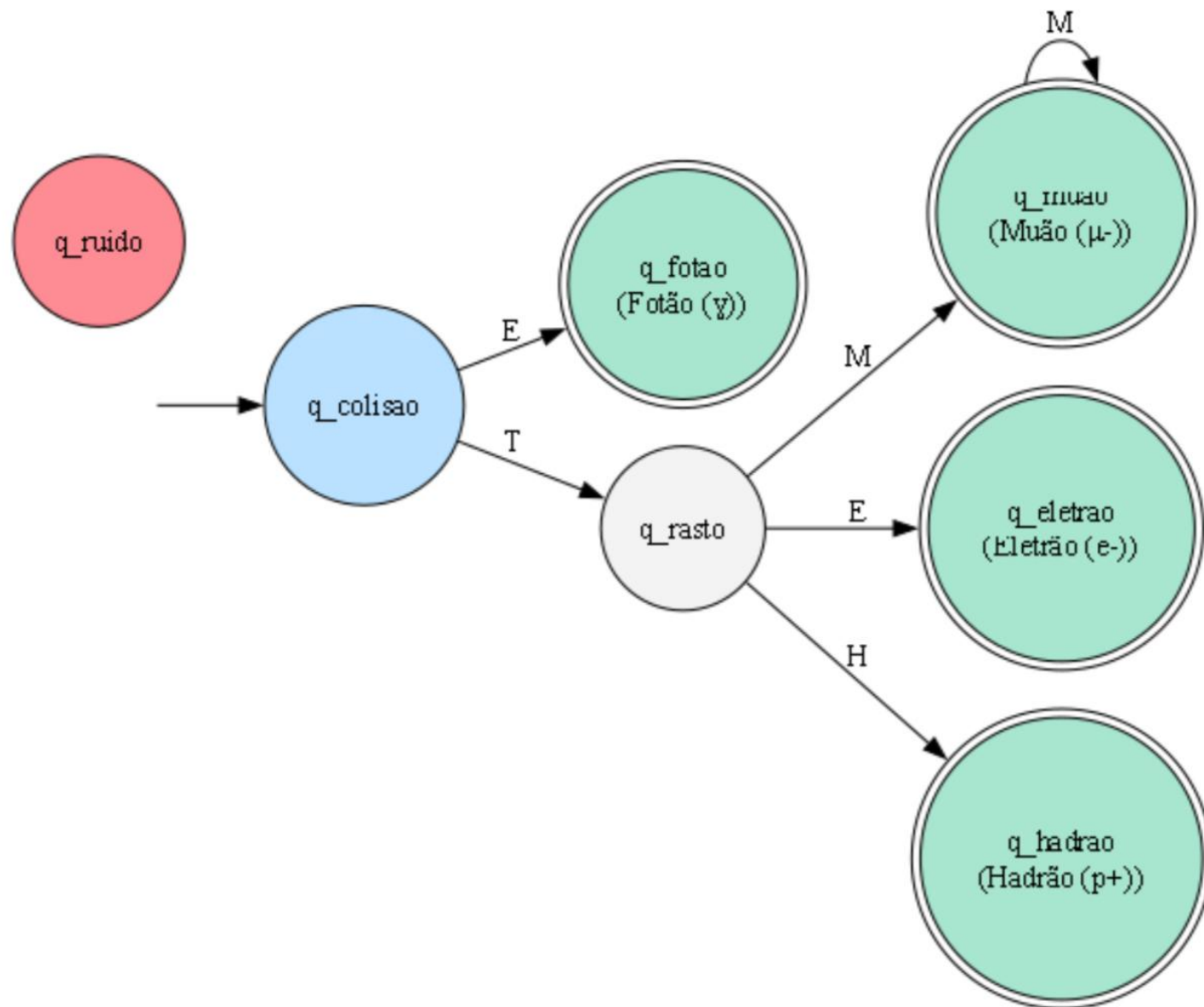
    if estado_atual in self.estados_aceitacao:
        partícula = self.estados_aceitacao[estado_atual]
        if not modo_silencioso:
            print(f"Aceite. corresponde a um {partícula}.")
        return True
    else:
        if not modo_silencioso:
            print("Rejeitado, é apenas ruído.")
        return False
```

analisar_assinatura(palavra) – função responsável por percorrer a sequência de sensores da palavra introduzida e verificar se a assinatura é válida segundo as regras definidas pelo autómato.



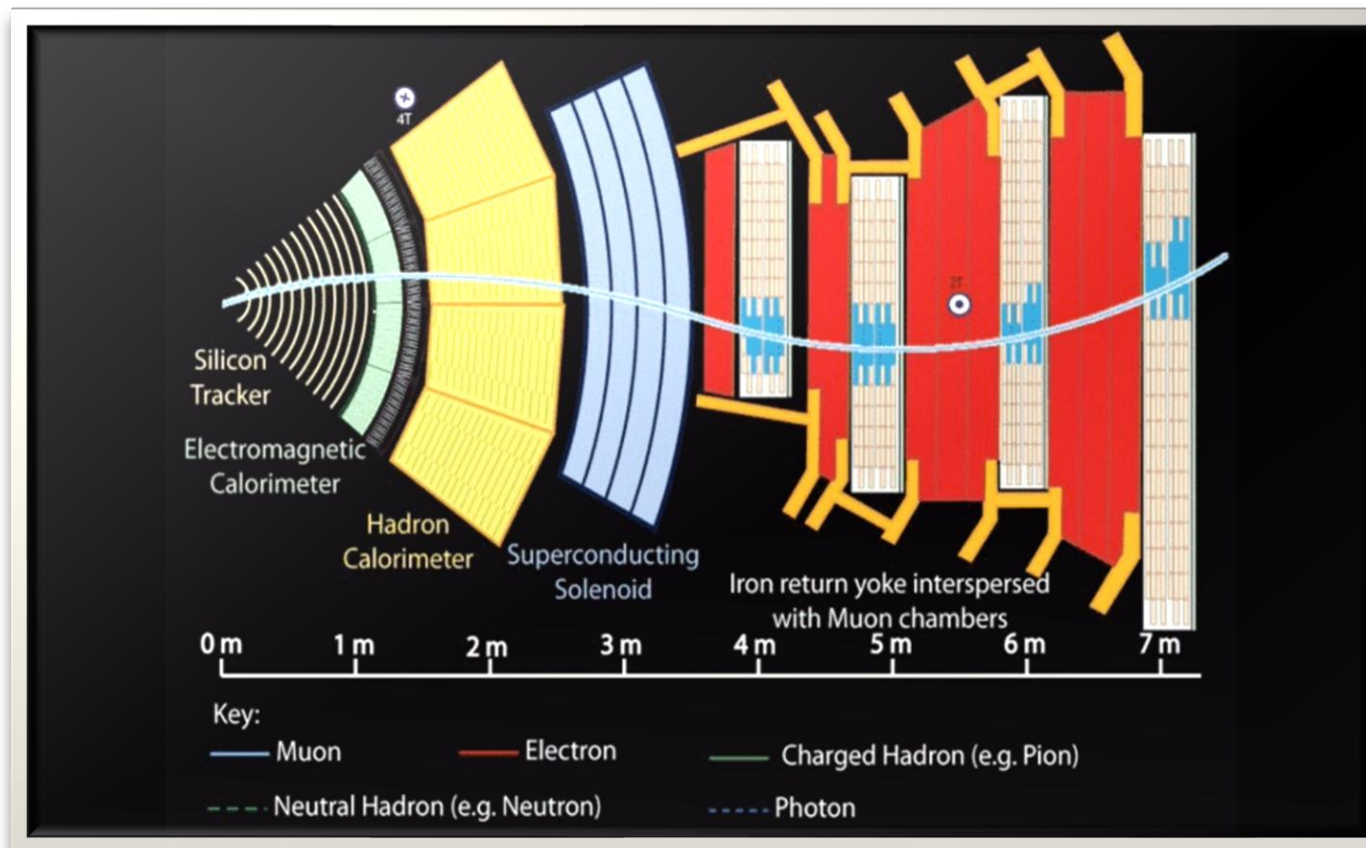
Código

Funcionamento



Conclusão

Este projeto permitiu demonstrar a aplicação prática dos autómatos finitos, evidenciando a sua eficiência em processos de validação e classificação.





Referências

- [1] CMS Collaboration, CERN. How CMS detects particles. <https://cms.cern/news/how-cms-detects-particles>
- [2] W3Schools. Python Tutorial. <https://www.w3schools.com/python/>
- [3] TutorialsPoint. Automata Theory Tutorial. https://www.tutorialspoint.com/automata_theory/index.htm